

Padrões de Projeto (Strategy)

QXD0068 - Reuso de Software

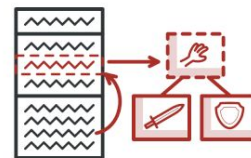
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Strategy
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



Strategy

Permite que você defina uma família de algoritmos, coloque-os em classes separadas, e faça os objetos deles intercambiáveis.

Introdução

- O Strategy (ou Estratégia) é um padrão de projeto comportamental que tem como objetivo definir uma família de algoritmos, encapsulá-los e torná-los intercambiáveis dentro de um mesmo contexto.
- Em outras palavras, ele permite que o comportamento de um objeto possa mudar em tempo de execução, sem que seja necessário modificar o código do próprio objeto.
- Um app de rotas pode usar diferentes estratégias:
 -  carro |  a pé |  bicicleta

Padrão Strategy

- O Padrão Strategy permite encapsular algoritmos relacionados em classes separadas, tornando-os intercambiáveis em tempo de execução.
- O objetivo é separar o comportamento (a lógica do algoritmo) do contexto que o utiliza, favorecendo a flexibilidade e a extensibilidade do código.
- Em vez de usar condicionais como if ou when para decidir o algoritmo, o contexto delega a execução para uma estratégia específica.

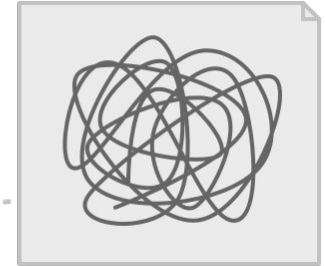
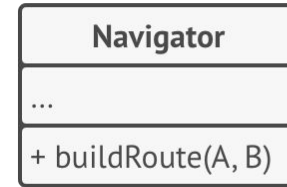


Problema

- Uma aplicação de navegação foi criada para ajudar viajantes a encontrar rotas em um mapa. Inicialmente, ela gerava apenas rotas para carros, mas, com o tempo, foram adicionadas novas opções — rotas para pedestres, transporte público, ciclistas e pontos turísticos.
- Com isso, o código da aplicação cresceu demais: a classe principal ficou enorme, difícil de manter e de modificar. Qualquer ajuste em um algoritmo de rota afetava toda a estrutura, aumentando o risco de erros. Além disso, o desenvolvimento em equipe se tornou problemático, pois todos precisavam alterar o mesmo código central, gerando conflitos e baixa produtividade.
- Em resumo, o problema principal é o acoplamento excessivo entre os algoritmos de roteamento e a classe principal, o que causa baixa manutenção, pouca extensibilidade e dificuldade de colaboração.

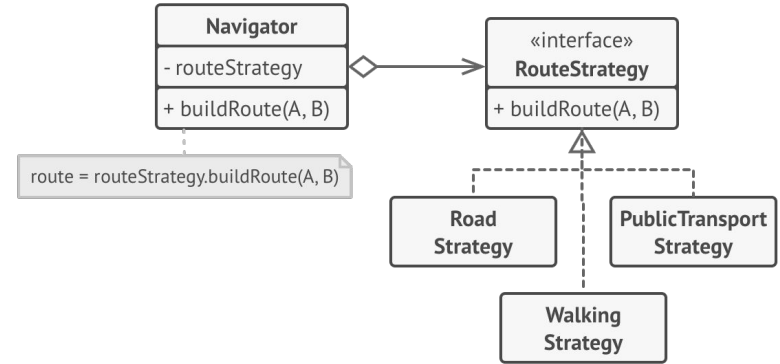
Problema

- **Cenário:**
 - Um app de navegação que gera rotas.
 - Inicialmente, apenas “rota de carro”.
 - Depois: “a pé”, “ônibus”, “bicicleta”...
- **Problema:**
 - A classe principal cresceu demais.
 - Cada novo algoritmo duplicava código.
 - Difícil de manter e testar.
- **Sintoma:**
 - “Muitos if/else para decidir qual algoritmo usar.”



Solução

- **Ideia central do Strategy:**
 - Extrair cada algoritmo para uma classe separada.
 - Criar uma interface comum (Strategy).
 - O Contexto mantém uma referência para uma estratégia ativa.
 - O Cliente escolhe qual usar em tempo de execução.
- **Resultado:**
 - O contexto não conhece detalhes de cada algoritmo.
 - É possível adicionar novos algoritmos sem tocar no código existente.



Analogia do Mundo Real

- Imagine que você precisa chegar ao aeroporto:
 - Pode ir de ônibus, táxi, ou bicicleta.
 - Cada meio é uma estratégia de transporte.
 - Você escolhe uma conforme o tempo ou orçamento.
- O destino (contexto) é o mesmo; apenas a estratégia muda.

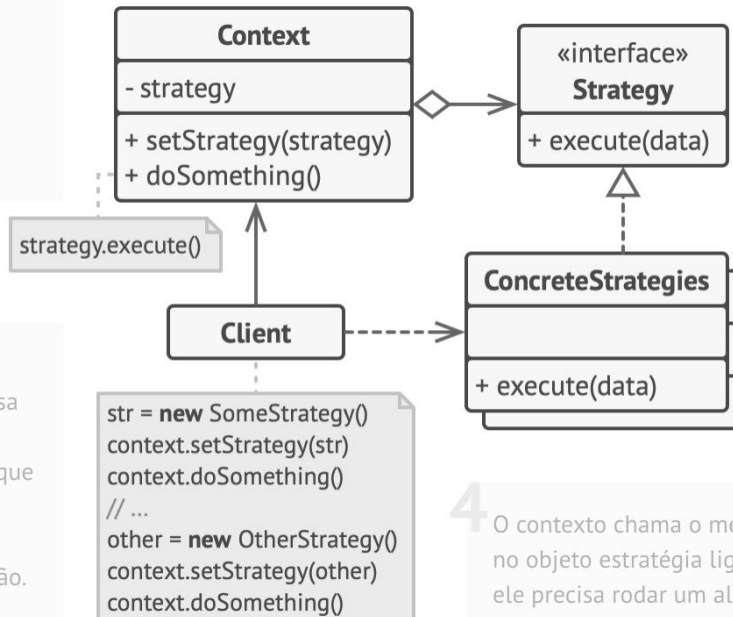
Analógia do Mundo Real



Estrutura do Padrão

- Participantes:
 - Strategy (interface): define o método comum.
 - ConcreteStrategy: implementa uma variação do algoritmo.
 - Context: usa a estratégia escolhida para executar a ação.
 - Client: seleciona qual estratégia será usada.

1 O **Contexto** mantém uma referência para uma das estratégias concretas e se comunica com esse objeto através da interface da estratégia.



2 A interface **Estratégia** é comum à todas as estratégias concretas. Ela declara um método que o contexto usa para executar uma estratégia.

3 **Estratégias Concretas** implementam diferentes variações de um algoritmo que o contexto usa.

4 O contexto chama o método de execução no objeto estratégia ligado cada vez que ele precisa rodar um algoritmo. O contexto não sabe qual tipo de estratégia ele está trabalhando ou como o algoritmo é executado.

5 O **Cliente** cria um objeto estratégia específico e passa ele para o contexto. O contexto expõe um setter que permite o cliente mudar a estratégia associada com contexto durante a execução.

```
interface Operacao {
    fun calcular(a: Int, b: Int): Int
}

class Soma : Operacao { override fun calcular(a, b) = a + b }
class Subtracao : Operacao { override fun calcular(a, b) = a - b }

class Calculadora(private var operacao: Operacao) {
    fun setOperacao(op: Operacao) { operacao = op }
    fun executar(a: Int, b: Int) = operacao.calcular(a, b)
}

fun main() {
    val calc = Calculadora(Soma())
    println(calc.executar(10, 5)) // 15
    calc.setOperacao(Subtracao())
    println(calc.executar(10, 5)) // 5
}
```

Exemplo



```
from abc import ABC, abstractmethod

class Operacao(ABC):
    @abstractmethod
    def calcular(self, a, b): pass

class Soma(Operacao):
    def calcular(self, a, b): return a + b

class Subtracao(Operacao):
    def calcular(self, a, b): return a - b

class Calculadora:
    def __init__(self, operacao): self.op = operacao
    def set_operacao(self, op): self.op = op
    def executar(self, a, b): return self.op.calcular(a, b)

calc = Calculadora(Soma())
print(calc.executar(10, 5)) # 15
calc.set_operacao(Subtracao())
print(calc.executar(10, 5)) # 5
```

Exemplo



Aplicabilidade

- **Use Strategy quando:**
 - Você tem várias variações de um mesmo algoritmo.
 - Quer trocar comportamentos em tempo de execução.
 - Deseja eliminar estruturas condicionais complexas.
 - Precisa isolar regras de negócio de implementações específicas.

Prós e Contras

- **Vantagens:**

- Substitui if/else por polimorfismo.
- Aderente ao Princípio Aberto/Fechado (OCP).
- Facilita testes e extensões.
- Trocável em tempo de execução.

- **Desvantagens:**

- Aumenta o número de classes.
- Cliente deve conhecer as estratégias.
- Pode ser desnecessário para poucos algoritmos.

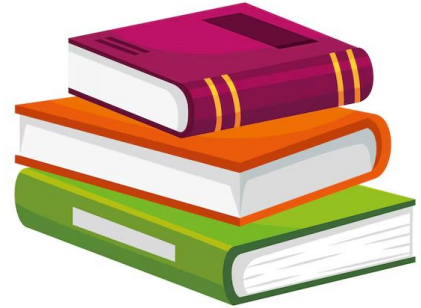
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

